

Coding Flow Explanation

What this coding module does

This coding module lets an admin create coding tracks, lets a student solve them in the browser, runs sample test cases on the code, and then gives a final evaluation.

It works in 4 main parts:

- 1 Coding track generation
- 1 Coding workspace and sample test execution
- 1 Final AI evaluation
- 1 Saving and showing the result

1. Coding track generation

Where it starts

The admin uses the Coding Track Studio page:

- 1 ``src/app/admin/coding/page.tsx``

What the admin enters

The admin can enter:

- 1 role name
- 1 topic list
- 1 difficulty
- 1 language
- 1 duration
- 1 optional Gemini API key

What happens next

When the admin clicks **Generate Coding Track**:

- 1 The frontend sends a request to the Gemini API route.
- 1 The route uses the action ``generate_coding_track``.
- 1 Gemini returns JSON for the coding track.

- 1 That JSON is loaded back into the admin form.
- 1 The admin can review it and save it.

What the generated coding track contains

A coding track stores:

- 1 title
- 1 role name
- 1 topics
- 1 difficulty
- 1 duration
- 1 language
- 1 problem statement
- 1 starter code
- 1 function name
- 1 sample tests
- 1 evaluation criteria
- 1 expected strong-solution signals

Important rule for sample tests

To reduce runner errors, sample tests are generated in a safe format:

- 1 one array argument: ``[1,2,3]``
- 1 one number argument: ``5``
- 1 multiple arguments: ``{"args":[[1,2,3],3]}``
- 1 one string argument: ``"hello"``

This logic is guided from:

- 1 ``src/app/api/gemini/route.ts``

2. Saving the coding track

When the admin clicks ****Save Coding Track****:

- 1 The form checks that required fields are present.

- 1 The sample tests are parsed and normalized.
- 1 The track is saved using the data service.

Where it is saved

The app saves coding tracks through:

- 1 ``src/lib/data-service.ts``

This service supports:

- 1 Firebase, if Firebase is available
- 1 local storage, as fallback

So the app can still work even if Firebase is not active.

3. Student coding workspace

Where the student solves the problem

The student opens:

- 1 ``src/app/coding/[trackId]/page.tsx``

What the page shows

The coding workspace shows:

- 1 coding problem
- 1 function name to complete
- 1 sample tests
- 1 Monaco code editor
- 1 explanation box
- 1 optional evaluation API key box

Why Monaco is used

Monaco is the browser editor used in VS Code-like interfaces.

It is useful here because it gives:

- 1 code editing experience
- 1 syntax highlighting
- 1 language support
- 1 cleaner coding UI

4. Running sample test cases

What happens when the student clicks Run Sample Tests

- 1 The page collects:
 - 1 language
 - 1 function name
 - 1 student code
 - 1 sample tests
- 1 It sends them to:
 - 1 ``/api/submit``
- 1 That API calls the execution engine.
- 1 The execution engine writes temporary files.
- 1 It compiles or runs the code.
- 1 It executes the sample tests.
- 1 It returns:
 - 1 output
 - 1 errors
 - 1 execution time
 - 1 pass/fail result for each test

API route used

- 1 ``src/app/api/submit/route.ts``

Execution engine used

- 1 ``src/lib/server/code-submit.ts``

5. How test case execution works internally

Step A: input reaches the runner

The runner receives:

- 1 language
- 1 function name
- 1 code
- 1 tests

Each test has:

- 1 id
- 1 input
- 1 expected

Step B: input is normalized

The runner tries to convert test input into usable arguments.

Examples:

- 1 ``[1,2,3]`` becomes one array argument
- 1 ``5`` becomes one number argument
- 1 ``{"args":[[1,2,3],3]}`` becomes two arguments
- 1 ``"hello"`` becomes one string argument

This normalization is important because different questions need different argument types.

Step C: language-specific harness is built

The system adds extra harness code around the student code.

That harness:

- 1 finds the target function
- 1 loops through the sample tests
- 1 calls the function with parsed arguments
- 1 compares actual result with expected result
- 1 creates a JSON result payload

Step D: execution by language

Python

- 1 writes a `.py`` file
- 1 runs it with ``python``

JavaScript

- 1 writes a `.mjs`` file
- 1 runs it with ``node``

C

- 1 writes a `.c`` file
- 1 compiles with ``gcc``
- 1 runs the compiled executable

C++

- 1 writes a `.cpp`` file
- 1 compiles with ``g++``
- 1 runs the compiled executable

C++ runner details

The C++ runner adds support such as:

- 1 ``#include``
- 1 ``using namespace std;``
- 1 ``ll`` and ``llong`` aliases
- 1 ``gnu++17`` compilation mode

It also supports common coding-platform styles like:

- 1 free function style
- 1 ``class Solution`` style

Safety and control inside execution

The runner also has some basic control features:

- 1 temporary folder per run
- 1 timeout handling
- 1 error capture
- 1 compile error reporting
- 1 runtime error reporting
- 1 final cleanup of temp files

6. What the student sees after running sample tests

The workspace shows:

- 1 how many tests passed
- 1 expected output
- 1 actual output
- 1 per-test error, if any
- 1 execution output
- 1 execution errors
- 1 execution time in milliseconds

This gives the student a quick check before final submission.

7. Final submission and AI evaluation

What happens when the student clicks Submit For Evaluation

- 1 The student code is collected.
- 1 The explanation text is collected.
- 1 A summary of the sample test results is created.
- 1 All of that is sent to the Gemini route with:
- 1 ``evaluate_code_submission``
- 1 Gemini returns evaluation JSON.
- 1 The app parses the score and feedback.
- 1 The final attempt is saved.
- 1 The user is redirected to the coding result page.

What the AI evaluates

The AI is asked to judge:

- 1 problem understanding
- 1 likely correctness
- 1 edge case handling
- 1 code quality
- 1 explanation quality

What the AI returns

The evaluation stores:

- 1 score
- 1 strengths
- 1 weaknesses
- 1 suggestions
- 1 summary

This logic is in:

- 1 ``src/app/api/gemini/route.ts``

8. Difference between sample test execution and final evaluation

This is very important.

Sample test execution

This is execution-based.

It checks:

- 1 whether the code runs
- 1 whether sample tests pass
- 1 compile errors
- 1 runtime errors
- 1 output mismatch

Final AI evaluation

This is rubric-based.

It checks:

- 1 code quality
- 1 explanation quality
- 1 problem understanding
- 1 likely completeness

So the final score is not only based on sample tests.

It also depends on the quality of the solution and explanation.

9. Saving the final coding attempt

After evaluation, the app saves a coding attempt with fields like:

- 1 track id
- 1 student info
- 1 code
- 1 explanation
- 1 score
- 1 strengths
- 1 weaknesses
- 1 suggestions
- 1 evaluation summary
- 1 sample test summary
- 1 sample test results
- 1 number of passed tests
- 1 total tests
- 1 timestamps

This is saved through:

- 1 ``src/lib/data-service.ts``

The structure is defined in:

1 `src/types/models.ts`

10. Coding result page

Where the result is shown

1 `src/app/coding/result/[attemptId]/page.tsx`

What the result page displays

The result page shows:

- 1 score
- 1 passed sample tests
- 1 difficulty
- 1 language
- 1 summary
- 1 strengths
- 1 weaknesses
- 1 suggestions
- 1 sample test run summary
- 1 executed sample test details
- 1 submitted code

The feedback is shown point-wise so it is easier to read.

11. API key fallback system

The Gemini route supports:

- 1 one API key
- 1 multiple API keys
- 1 optional key entered in UI
- 1 server-side fallback keys

If one key fails, the route can try the next key.

This helps avoid failures when one key reaches limit or quota.

12. Beginner-friendly full flow summary

Here is the full coding flow in simple order:

- 1 Admin opens Coding Track Studio.
- 1 Admin enters role, topics, difficulty, language, and duration.
- 1 Gemini generates a coding problem with starter code and sample tests.
- 1 Admin reviews and saves the coding track.
- 1 Student opens the coding track.
- 1 Student writes code in Monaco editor.
- 1 Student runs sample tests.
- 1 The backend compiles or runs the code.
- 1 The backend compares actual output with expected output.
- 1 The app shows test results, errors, and execution time.
- 1 Student submits code with explanation.
- 1 Gemini performs final rubric-based evaluation.
- 1 The app saves the attempt.
- 1 The result page shows short point-wise feedback.

13. Important practical note

The current system supports real sample-test execution for:

- 1 Python
- 1 JavaScript
- 1 C
- 1 C++

The final AI evaluation is useful, but it is still different from a full online judge with hidden test cases.

So the current design is best described as:

- 1 execution-backed sample testing
- 1 AI-backed final review

14. Main files involved